

HavenWorld Build Guide — Part 10B of 10

SERIES FINALE — Open-Source Community, Volunteer Moderators, Phase 4+ Roadmap, Ethical Monetization, GDPR & Complete Series Reference

PART 10B OF 10 Author: Daniel | Anoka, MN | Friday, September 18, 2026
| HavenWorld Indie Game Project



You made it, Daniel.

This is Part 10B — the final document of the entire HavenWorld Build Guide series. Over 10 parts, you've gone from a blank terminal to a fully deployed, feature-rich 3D social virtual world running on 100% free infrastructure. This document covers the community layer, the volunteer moderator program, the Phase 4+ roadmap, ethical monetization design, GDPR compliance, and a complete series reference you'll return to again and again. The blueprint is finished. The world is yours.

Document scope: This guide assumes you have completed Parts 1 through 10A. HavenWorld is deployed, monitored, and scaling. This final part transforms your solo project into a sustainable open-source community game with the infrastructure, documentation, and roadmap to grow beyond its first release.

Section 1 — Open-Source

Community Health

A game is only as alive as its community. Open-sourcing HavenWorld invites contributors who fix bugs faster than you can, add features you never imagined, and extend the game's lifespan indefinitely. But open-source communities don't run themselves — they need documentation, processes, and norms. This section provides everything ready to paste into your repository.

1.1 CONTRIBUTING.md

Place this file at the repository root. It is the first thing a new contributor reads after the README.

```
# Contributing to HavenWorld Thank you for contributing! HavenWorld is a
solo-to-community project and every contribution – bug fix, new feature,
documentation improvement, or typo fix – makes the game better for everyone.
--- ## Quick Start 1. **Fork** the repository on GitHub. 2. **Clone** your
fork locally: git clone https://github.com/YOUR-USERNAME/havenworld.git
cd havenworld 3. **Add upstream remote:** git remote add upstream
https://github.com/havenworld/havenworld.git 4. **Create a feature branch
from `develop`** (never from `main`): git checkout develop git pull
upstream develop git checkout -b feature/fishing-tournament --- ##
Branch Naming Use one of the following prefixes: feature/fishing-
tournament # New Features fix/socket-disconnect # Bug
fixes docs/avatar-pipeline # Documentation only
refactor/room-manager # Code restructuring, no behaviour change
test/crafting-unit # Test additions or fixes chore/update-
dependencies # Maintenance, tooling --- ## Local Development Setup
[MAC] Install dependencies: pnpm install [MAC] Start the full local dev
environment (client + server hot-reload): pnpm dev [MAC] Run all tests
before opening a PR: pnpm test [MAC] Type-check (must pass – zero
errors): pnpm tsc --noEmit [MAC] Lint (must pass – zero errors): pnpm
lint --- ## Commit Message Format We use Conventional Commits. Format:
<type>(<scope>): <short description> Examples: feat(fishing): add
tournament bracket system fix(socket): resolve disconnect race condition
on room join docs/avatar): update Blender pipeline guide for 4.x
refactor(room): extract RoomManager to separate service test(crafting):
add unit tests for recipe validation chore(deps): bump babylonjs to 7.14.0
Types: feat, fix, docs, refactor, test, chore, perf, ci --- ## Pull Request
Checklist Before requesting review, verify every item: [ ] All existing
tests pass (`pnpm test`) [ ] New feature or fix includes tests [ ] No
```

```
TypeScript errors (`pnpm tsc --noEmit`)    [ ] No ESLint errors (`pnpm lint`)
[ ] PR title follows Conventional Commits format    [ ] PR description
explains WHAT changed and WHY, not just what    [ ] Screenshots or screen
recordings included for UI changes    [ ] Breaking changes are called out
clearly in the description --- ## What Makes a Good PR Description Bad:
"Fixed the bug" Good: "Fix: Socket.io race condition when two players join
the same room simultaneously. The RoomManager.addPlayer() method was
not awaiting the Redis lock, causing duplicate player entries. Added
distributed lock via ioredis SETNX." --- ## Code Style - TypeScript strict
mode is enforced - no `any` types - Prettier handles formatting (runs
automatically on commit via husky) - ESLint rules are project-defined - do
not disable rules without discussion - All shared game types live in
`packages/shared/src/types/` --- ## Questions? Open a Discussion on GitHub
- not an Issue. Issues are for bugs and accepted feature requests only.
Discussions are for questions, ideas, and feedback. Thank you for helping
build HavenWorld. 🏡
```

1.2 Good-First-Issue Label Guide

A healthy GitHub label system is the on-ramp for new contributors. Apply these labels consistently — they signal intent and help contributors self-sort by confidence level.

Label	Color (Hex)	Meaning	Example Issue
good first issue	#7ee8a2	Isolated, self-contained, well-documented. No architectural knowledge needed.	"Add a new fish species to the fishing catalog"
help wanted	#0e7c7b	Known solution exists, medium complexity. Author welcomes external PRs.	"Add sound effect for furniture placement"
bug	#d73a4a	Confirmed broken behaviour. Reproduction steps must be included in the issue body.	"Avatar falls through floor mesh on room load"
enhancement	#a2eeef	Accepted feature request. Design is approved; implementation is	"Add pet hunger decay over time"

Label	Color (Hex)	Meaning	Example Issue
		open.	
docs	#e4b400	Documentation improvement — README, JSDoc, wiki, or guide.	"Document Socket.io event payload types"
wontfix	#ffffff	Out of scope or rejected. A comment MUST explain why before closing.	"Add Minecraft-style crafting"

How to Write a Good First Issue

A good first issue removes all ambiguity so a brand-new contributor can start immediately. Include:

1. **Context:** One paragraph explaining why this exists in the codebase.
2. **Expected result:** Exactly what "done" looks like.
3. **Acceptance criteria:** Checklist of things a reviewer will verify.
4. **File pointers:** Tell them exactly where to look. Example: *"Edit the catalog array in packages/shared/src/fishing.ts — each fish entry follows the FishSpecies type defined at the top of the file."*
5. **Testing note:** What test to run or write to verify the change.

Note:

A first issue should take a motivated beginner no more than 2 hours. If the scope creeps beyond that, split it into smaller issues.

1.3 Issue Templates

Place these files in `.github/ISSUE_TEMPLATE/`. GitHub will automatically present them as options when a user clicks "New Issue."

Bug Report Template — `.github/ISSUE_TEMPLATE/bug_report.yml`

```
name: Bug Report description: File a bug report title: "[BUG] " labels:
["bug"] assignees: [] body: - type: markdown attributes: value:
| Thank you for reporting a bug! Please fill out all fields as
completely as possible. Issues without reproduction steps will be
closed. - type: textarea id: description attributes: label:
Bug Description description: A clear and concise description of what
the bug is. placeholder: "When I click the fishing rod button, the game
freezes." validations: required: true - type: textarea id:
steps attributes: label: Steps to Reproduce description:
Step-by-step instructions to reproduce the issue. value: | 1.
Go to the Fishing Pier room 2. Click the fishing rod icon in the
toolbar 3. Wait 3 seconds 4. Observe freeze / error
validations: required: true - type: textarea id: expected
attributes: label: Expected Behaviour description: What should
happen? placeholder: "The fishing minigame panel should open."
validations: required: true - type: textarea id: actual
attributes: label: Actual Behaviour description: What actually
happens? placeholder: "The browser tab freezes and must be refreshed."
validations: required: true - type: dropdown id: os
attributes: label: Operating System options: - Windows 11
- Windows 10 - macOS Sonoma - macOS Ventura -
Ubuntu / Debian - iOS (iPhone / iPad) - Android -
Other validations: required: true - type: dropdown id:
browser attributes: label: Browser options: -
Chrome / Chromium - Firefox - Safari - Edge -
Other validations: required: true - type: input id: version
attributes: label: HavenWorld Version / Commit placeholder: "e.g.
v1.2.0 or commit abc1234" validations: required: true - type:
textarea id: screenshots attributes: label: Screenshots /
Console Errors description: Paste any browser console errors or attach
screenshots. validations: required: false
```

Feature Request Template —

`.github/ISSUE_TEMPLATE/feature_request.yml`

```
name: Feature Request description: Suggest a new feature or improvement
title: "[FEATURE] " labels: ["enhancement"] assignees: [] body: - type:
markdown attributes: value: | Thanks for suggesting a
feature! Please provide as much context as possible so we can
evaluate it properly. - type: textarea id: problem attributes:
label: Problem Being Solved description: | What problem does
this feature address? Don't describe the feature yet — describe the
underlying need. placeholder: | "As a player, I have no way to
express emotion in the game beyond text chat. This makes interactions
feel flat." validations: required: true - type: textarea id:
solution attributes: label: Proposed Solution description:
Describe your idea clearly. placeholder: "Add an emoji reaction toolbar
that plays a short avatar animation." validations: required: true
- type: textarea id: alternatives attributes: label:
Alternatives Considered description: What other solutions did you think
about? placeholder: "I considered using chat commands like /wave but a
toolbar is more discoverable." validations: required: false -
```

```
type: textarea      id: context      attributes:      label: Additional
Context            description: Screenshots, mockups, references to similar games,
or any extra detail.      validations:      required: false
```

1.4 PR Checklist Template

Place this file at `.github/pull_request_template.md`. It auto-populates every new PR description.

```
## Summary <!-- Explain WHAT changed and WHY, not just what. Include a link
to the      related issue if one exists. --> Closes # <!-- issue number -->
--- ## Type of Change - [ ] Bug fix (non-breaking change that resolves an
issue) - [ ] New feature (non-breaking change that adds functionality) - [ ]
Breaking change (fix or feature that would cause existing functionality to
not work as expected) - [ ] Documentation update - [ ] Refactor (no behaviour
change) - [ ] Performance improvement - [ ] Test addition or fix --- ##
Testing <!-- Describe how you tested this change. --> - [ ] All existing
tests pass (`pnpm test`) - [ ] New tests added for this change - [ ] Manual
testing performed – describe steps below:  **Manual test steps:** 1. 2. 3.
--- ## Code Quality - [ ] No TypeScript errors (`pnpm tsc --noEmit`) - [ ]
No ESLint errors (`pnpm lint`) - [ ] No new `console.log` statements left in
production code - [ ] No hardcoded credentials, tokens, or secrets --- ##
UI Changes - [ ] N/A – no UI changes in this PR - [ ] Screenshots or screen
recording attached below --- ## Breaking Changes <!-- If this is a breaking
change, describe what breaks and what users need to do. --> None / [describe
breaking change here] --- ## Reviewer Notes <!-- Anything specific you'd
like the reviewer to pay attention to? -->
```

1.5 CODE_OF_CONDUCT.md

Based on the Contributor Covenant v2.1, adapted for a friendly gaming community.
Place at the repository root.

Our Pledge

We as members, contributors, and maintainers pledge to make participation in HavenWorld a harassment-free experience for everyone, regardless of age, body size, visible or invisible disability, ethnicity, sex characteristics, gender identity and expression, level of experience, education, socioeconomic status, nationality, personal appearance, race, caste, color, religion, or sexual identity and orientation.

Standards of Behaviour

Encouraged behaviours include: using welcoming and inclusive language; being respectful of differing viewpoints and experience; gracefully accepting constructive criticism; focusing on what is best for the community; showing empathy toward other community members.

Unacceptable behaviours include: harassment, discrimination, or slurs in any form; trolling, insulting, or derogatory comments; personal or political attacks; publishing others' private information (doxxing); sexual language or imagery in any project space; sustained disruption of discussion.

Enforcement

Step	Trigger	Action
1 — Warning	First minor violation	Private written warning with explanation of the violation
2 — Temporary Ban	Repeated violations or single serious violation	Temporary ban from all community spaces (7-30 days)
3 — Permanent Ban	Severe violation or continued behaviour after temporary ban	Permanent ban from all community spaces — no appeal

Reporting: Report violations by emailing `havenworld-conduct@[maintainer]`. All reports will be reviewed within 48 hours and kept confidential. The reporting process will never expose the identity of the reporter without consent.

1.6 SECURITY.md

Place this file at the repository root. GitHub surfaces it automatically on the Security tab.

```
# Security Policy – HavenWorld  ## Reporting a Vulnerability  **PLEASE DO NOT
open a public GitHub Issue for security vulnerabilities.**  Publicly
disclosing a vulnerability before it is patched puts all players at risk. Use
```

one of the following private disclosure methods instead: 1. **Email:** Send a detailed report to [maintainer-private-email@domain.com] (replace with your actual contact email) 2. **GitHub Private Security Advisory:** Go to Security > Advisories > "Report a vulnerability" on this repository. GitHub keeps this conversation private until a patch is released. --- ##

Response Timeline	Phase	SLA
Within 48 hours	Status update	Acknowledgement
Patch (Critical)	Within 14 days of report	Within 7 days
Within 30 days of report	Patch (Medium/Low)	Patch (High)
Next scheduled release		

After a patch is released, we will publicly credit the reporter (if they wish) in the release notes and SECURITY.md changelog. --- ##

In-Scope Vulnerabilities The following vulnerability classes are in scope for responsible disclosure:

- Authentication bypass (JWT forgery, session fixation, privilege escalation)
- SQL injection via Prisma raw queries
- Cross-site scripting (XSS) – stored or reflected
- Socket.io session hijacking or unauthorized room access
- Cloudflare R2 access control bypass (reading/writing other users' assets)
- Insecure direct object reference (IDOR) on account or inventory endpoints
- Denial of service via Socket.io event flooding (if bypassing rate limiter)
- Server-side request forgery (SSRF) via UGF upload endpoint

--- ##

Out-of-Scope The following are NOT eligible for responsible disclosure:

- Rate limiting annoyances that do not result in data exposure
- Self-XSS (requires the attacker to execute scripts in their own browser)
- Issues requiring physical access to a user's device
- Social engineering attacks on the moderation team
- Outdated dependency version reports without a working proof of concept
- Issues in third-party services (Cloudflare, Oracle, Discord) – report those upstream

--- ##

Supported Versions	Version	Supported
Latest	Yes (always)	
Previous	Security fixes only	Older
		No

Section 2 — Volunteer Moderator Program

HavenWorld needs human moderators before it launches — not after a crisis. A lightweight, well-documented volunteer program protects players, supports community growth, and prevents burnout for you as the solo developer. This section gives you the full technical and operational stack.

2.1 In-Game Moderator Rank

The `role` field on the `User` table (defined in your Prisma schema in Part 4A) carries three values: `PLAYER`, `MOD`, and `ADMIN`. This single field gates all moderation functionality.

Moderator Badge in Babylon.js

When rendering a player's nameplate in the 3D scene, check the player's role from the Socket.io `ROOM_JOIN` payload and render a gold shield mesh above the name if `role === 'MOD'` or `role === 'ADMIN'`. The shield is a simple `MeshBuilder.CreateBox` scaled small and tinted `#FFD700` via a `StandardMaterial`.

In-Game Moderator Chat Commands

Command	Effect	Minimum Role
<code>/mute @username 10m</code>	Mutes player for 10 minutes (no chat messages processed)	MOD
<code>/warn @username message</code>	Sends formal warning to player, logs to ModLog	MOD
<code>/kick @username</code>	Disconnects player from server (can immediately rejoin)	MOD
<code>/ban @username reason</code>	Permanently bans player, disconnects immediately	ADMIN
<code>/modlog @username</code>	Shows mod action history for player (MOD channel only)	MOD

Server-Side Moderation Handler

[MAC/LINUX] Create file: `apps/server/src/handlers/moderation.ts`

```
// apps/server/src/handlers/moderation.ts
import { prisma } from '../lib/prisma';
import { redis } from '../lib/redis';
import { io } from '../index';

export function registerModerationHandlers(socket: AuthSocket) {
  socket.on('MOD_ACTION', async ({ action, targetId, reason, duration }) => {
    // Verify caller is MOD or ADMIN – never trust the client
    const mod = await prisma.user.findUnique({
      where: { id: socket.data.userId },
      select: { id: true, username: true, role: true }
    });
    if (mod?.role !== 'MOD' && mod?.role !== 'ADMIN') {
      socket.emit('ERROR', { code: 'FORBIDDEN', message: 'Insufficient permissions' });
      return;
    }
    // ADMIN-only actions
    if (action === 'BAN' && mod.role !== 'ADMIN') {
      socket.emit('ERROR', { code: 'FORBIDDEN', message: 'BAN requires ADMIN' });
    }
  });
}
```

```

role' });      return;      }      switch (action) {      case 'MUTE':
// Store mute in Redis with TTL (duration is in minutes)      await
redis.setEx(`muted:${targetId}`, duration * 60, 'true');
io.to(targetId).emit('MUTED', { duration, reason });      break;
case 'WARN':      // Deliver warning in-game and persist it
io.to(targetId).emit('MOD_WARNING', { message: reason });      break;
case 'KICK':      // Disconnect the target socket immediately
const targetSocket = io.sockets.sockets.get(targetId);      if
(targetSocket) targetSocket.disconnect(true);      break;      case
'BAN':      // Persist ban to database and disconnect      await
prisma.user.update({      where: { id: targetId },      data: {
bannedAt: new Date(),      banReason: reason      }      });
const bannedSocket = io.sockets.sockets.get(targetId);      if
(bannedSocket) bannedSocket.disconnect(true);      break;      }      //
Audit log – every mod action is recorded, no exceptions      await
prisma.modLog.create({      data: {      modId: mod.id,      targetId,
action,      reason: reason ?? null,      duration: duration ?? null
}      });      // Notify the moderator that the action was applied
socket.emit('MOD_ACTION_CONFIRMED', { action, targetId });      }); }

```

Check on connection:

In your auth middleware, verify

`user.bannedAt !== null`

on every connection attempt. If banned, emit a

BANNED

event with the reason and disconnect — do not let the socket progress to room join.

2.2 Discord Ticket Bot (Free)

Player reports should not live in in-game chat where they can be seen by the accused. A private Discord ticket system gives reporters privacy and gives your mod team a trackable paper trail.

Setup

6. Create a Discord Application and bot at

discord.com/developers/applications.

7. Invite the bot to your HavenWorld Discord server with `Send Messages, Manage Channels, Embed Links, and Create Private Threads` permissions.
8. Create a category in Discord called `#mod-tickets`. Set it visible only to your mod team role.
9. Install dependencies: `npm install discord.js@14 dotenv`

Bot Core Logic (Simplified)

```
// discord-bot.js
const { Client, GatewayIntentBits, EmbedBuilder } =
require('discord.js');
const client = new Client({
  intents: [GatewayIntentBits.Guilds, GatewayIntentBits.GuildMessages]
});
client.on('interactionCreate', async (interaction) => {
  if (!interaction.isChatInputCommand()) return;
  if (interaction.commandName === 'report') {
    const reportedUser = interaction.options.getString('user');
    const reason = interaction.options.getString('reason');
    const room = interaction.options.getString('room') ?? 'Unknown';
    // Create a private thread in #mod-tickets
    const ticketChannel =
      interaction.guild.channels.cache.find(c => c.name === 'mod-tickets');
    const thread = await ticketChannel.threads.create({
      name: `report-${reportedUser}-${Date.now()}`,
      autoArchiveDuration: 10080, // 7 days
      reason: 'Player report'
    });
    const embed = new EmbedBuilder()
      .setTitle('🚨 Player Report')
      .setColor(0xd73a4a)
      .addFields(
        { name: 'Reporter', value: interaction.user.tag, inline: true },
        { name: 'Reported Player', value: reportedUser, inline: true },
        { name: 'In-Game Room', value: room, inline: true },
        { name: 'Reason', value: reason }
      )
      .setTimestamp();
    await thread.send({ embeds: [embed] });
    await interaction.reply({
      content: 'Report submitted. Our mod team has been notified.',
      ephemeral: true
    });
  }
});
client.login(process.env.DISCORD_BOT_TOKEN);
```

[LINUX] Deploy bot as a persistent PM2 process on your Oracle VM:

```
pm2 start discord-bot.js --name havenworld-discord-bot pm2 save pm2 startup
```

Cost:

\$0.00. The Discord bot runs within your existing Oracle VM free tier allocation alongside your HavenWorld server process. PM2 keeps it alive through crashes and reboots.

2.3 Escalation Ladder

Level	Trigger	Action	Handled By	Target Response Time
Level 1	Chat spam, minor rudeness, off-topic flooding	In-game mute (10–30 minutes)	Any Moderator	Immediate (on-duty mod)
Level 2	Harassment, slurs, scam attempt, sustained toxicity	24-hour mute + formal warning logged to ModLog	Senior Moderator	< 1 hour
Level 3	Repeat offender after Level 2, credible threats, coordinated griefing	7-day temporary ban + Admin review	Senior Mod + Admin	< 4 hours
Level 4	CSAM, doxxing, credible physical threats, large-scale fraud	Immediate permanent ban + report to relevant authorities / platform	Admin only	Immediate — no escalation needed

Level 4 Note:

Do not wait for a meeting or a vote on Level 4 violations. The Admin acts immediately and documents after the fact. Platform safety takes absolute priority over process.

| 2.4 Moderator Handbook (Summary)

Core Principles

- **Conflicts of interest:** Never moderate a situation involving a friend, a rival, or someone you have a personal dispute with. Recuse yourself and ping another mod.
- **Documentation first:** Take a screenshot before taking action. Post action + screenshot to the Discord mod channel within 5 minutes of the action.
- **ModLog is your paper trail:** Every action recorded in the modLog database table is permanent. This protects both players and moderators in disputes.
- **No power use in personal disputes:** Using mod tools for personal benefit is grounds for immediate removal from the program.
- **Consistency matters:** Apply rules the same way regardless of the player's social status, friend group, or fame in the community.

Operations

- Monthly mod team check-in in Discord (async summary post is acceptable — no mandatory voice call).
- Weekly availability expectation: maximum 8 hours/week. Moderating more than this causes burnout — communicate when you need a break.
- When leaving the program, give 2 weeks notice so a replacement can be recruited and trained.

Moderator Perks

Perk	Details
Exclusive Cosmetic Set	"HavenGuard" avatar outfit + furniture set — not available in shop
Monthly HavenGems Stipend	500 HavenGems/month credited automatically via server cron

Perk	Details
Early Feature Access	Access to all new features on staging server 2 weeks before public release
Mod Role in Discord	Visible moderator badge in the community Discord server

Section 3 — Phase 4+ Feature Roadmap

Everything built in Parts 1–10 is Phase 1–3. Phase 4+ is where HavenWorld evolves from "launched indie game" to "platform." The items below are sequenced by technical dependency and estimated effort. None require upfront cost — each builds on your existing free infrastructure.

3.1 Mobile App via Capacitor.js

Timeline: 6–12 months post-launch | **Platform lift:** High — dramatically widens your audience

Capacitor.js wraps your existing Vite PWA into native iOS and Android shells with minimal code changes. HavenWorld's architecture is already mobile-ready: Socket.io WebSockets work identically on mobile, and Babylon.js runs on mobile WebGL.

Installation and Setup

[MAC] Install Capacitor packages:

```
pnpm add @capacitor/core @capacitor/cli @capacitor/ios @capacitor/android
```

[MAC] Initialize Capacitor in your client package:

```
npx cap init HavenWorld com.havenworld.app --web-dir dist
```

[MAC] Build the client first:

```
pnpm --filter client build
```

[MAC] Add target platforms:

```
npx cap add ios npx cap add android
```

[MAC] Sync web assets to native platforms:

```
npx cap sync
```

[MAC] Open iOS project in Xcode (Mac only):

```
npx cap open ios
```

Key Adaptations Required

Area	Adaptation	Implementation
Avatar Movement	Virtual joystick for touch controls	Use <code>nipplejs</code> library overlaid on canvas
Furniture Interaction	Tap-to-interact replaces click	<code>Babylon.js pointerObservable</code> handles both pointer types
LOD Settings	Mobile-optimized mesh detail reduction	Detect <code>navigator.maxTouchPoints > 0</code> and apply low LOD preset
UI Layout	Larger touch targets, collapsed panels	CSS media query <code>max-width: 480px</code> — already partially done in PWA

Platform Costs (First Paid Costs in the Project)

Platform	Cost	Notes
iOS (Apple Developer Program)	\$99/year	Required to publish to App Store. Requires Mac + Xcode.
Android (Google Play)	\$25 one-time	One-time developer

Platform	Cost	Notes
		account registration fee.
Capacitor itself	Free (MIT license)	No ongoing cost for the framework.

3.2 🥽 WebXR / VR Mode

Timeline: 9–18 months post-launch | **Cost:** \$0 — WebXR is a web standard

Babylon.js has first-class native WebXR support built in. You do not need additional VR libraries. Your loft rooms become natural VR spaces — players can literally walk around their own apartment in virtual reality.

VR Manager Implementation

[MAC] Create file: `apps/client/src/engine/XRManager.ts`

```
// apps/client/src/engine/XRManager.ts
import { Scene, WebXRDefaultExperience } from '@babylonjs/core';
export async function initVR(scene: Scene): Promise<WebXRDefaultExperience | null> {
    // Check WebXR availability before attempting to create session
    const supported = await navigator.xr?.isSessionSupported('immersive-vr');
    if (!supported) return null;
    const xr = await scene.createDefaultXRExperienceAsync({
        floorMeshes: [scene.getMeshByName('floor')!],
        uiOptions: {
            sessionMode: 'immersive-vr',
            referenceSpaceType: 'local-floor'
        },
        teleportationOptions: {
            // Teleport locomotion prevents motion sickness
            floorMeshes: [scene.getMeshByName('floor')!]
        }
    });
    // Configure controller input mapping
    xr.input.onControllerAddedObservable.add((controller) => {
        controller.onMotionControllerInitObservable.add((motionController) => {
            // Map trigger to interaction, thumbstick to teleport target selection
            const trigger = motionController.getComponent('xr-standard-trigger');
            trigger?.onButtonStateChangedObservable.add((component) => {
                if (component.pressed) {
                    // Fire "interact" event – same as pointer click in flat mode
                    scene.pick(scene.pointerX, scene.pointerY);
                }
            });
        });
    });
    return xr;
}
// Call this after room load – show VR button only if hardware is present
export async function showVRButtonIfAvailable(scene: Scene) {
    const supported = await navigator.xr?.isSessionSupported('immersive-vr');
    if (supported) {
        document.getElementById('vr-enter-button')!.style.display = 'block';
        document.getElementById('vr-enter-button')!.onclick = () => initVR(scene);
    }
}
```

Compatible Hardware

Device	Browser	Notes
Meta Quest 2 / 3 / Pro	Meta Browser (built-in)	Best option — standalone, no PC needed
Steam VR headsets (Valve Index, Vive)	Chrome / Edge on PC	Requires PC + Steam VR runtime
PlayStation VR2	Chrome / Edge (via PS5 browser)	Limited browser; test thoroughly
Apple Vision Pro	Safari (visionOS)	WebXR supported since visionOS 1.1

3.3 🏠 Neighborhoods: Linked Personal Lofts

Timeline: 12-24 months post-launch | **Social lift:** Very high — encourages loft decoration and return visits

A shared "neighborhood street" room where player lofts appear as buildings on a stylized map. Players discover each other's spaces organically — walking past a house and choosing to knock.

Prisma Schema Addition

```
// Add to apps/server/prisma/schema.prisma
model Neighborhood {
  id          String @id @default(cuid())
  name        String // e.g. "Maple Street", "Crystal Boulevard"
  theme       String @default("suburban") // suburban | tropical | arctic | urban
  createdAt   DateTime @default(now())
  slots       LoftSlot[]
}
model LoftSlot {
  id          String @id @default(cuid())
  neighborhoodId String
  userId       String
  @unique      // one slot per user
  slotNumber   Int // position on the street map (1-20)
  displayName  String? // optional custom loft name
  createdAt    DateTime @default(now())
  neighborhood Neighborhood
  user         User
  @relation(fields: [neighborhoodId], references: [id])
  @relation(fields: [userId], references: [id])
  @@index([neighborhoodId, slotNumber])
}
```

[LINUX] Run migration after updating schema:

```
npx prisma migrate dev --name add-neighborhoods
```

Neighborhood Room Data Flow

10. Client requests neighborhood street room via `ROOM_JOIN { roomId: 'neighborhood:maple-street' }`.
11. Server calls `RoomManager.getNeighborhoodData(neighborhoodId)` — fetches all slots with user profiles, decoration scores, and visitor counts.
12. Room data sent to client as `NEIGHBORHOOD_LOADED { slots: LoftSlotWithMeta[] }`.
13. Client renders each slot as a building mesh. Clicking a building emits `ROOM_JOIN { roomId: 'loft:{userId}' }` — standard loft entry.

Access gating:

Neighborhood slot assignment requires VIP Tier 2+ (acquired via HavenGems spend threshold — not a separate purchase). This incentivizes organic gem spending without being pay-to-win.

3.4 🎨 User-Generated Furniture (UGF) System

Timeline: 18–30 months post-launch | **Engagement lift:** Extremely high — infinite content creation

Players upload custom Blender .glb models. With moderation, this transforms HavenWorld from a curated catalog into a living creative platform.

Upload Pipeline

14. Player exports .glb from Blender (enforced limits: $\leq 500\text{KB}$ uncompressed, $\leq 2,000$ triangles).
15. Client POSTs file to `/api/ugf/upload` with multipart form data.

16. Server validates: MIME type must be `model/gltf-binary`, size check, triangle count parsed via `@gltf-transform/core`.
17. **[LINUX]** Server applies Draco compression: `gltf-transform optimize model.glb output.glb --texture-compress none`
18. Model enters `ugf_pending` status — appears in admin moderation dashboard.
19. Mod team reviews (checks for inappropriate geometry, brand logos, NSFW content).
20. On approval: model uploaded to Cloudflare R2 at `/ugf/{userId}/{modelId}.glb`.
21. Player receives 50 HavenGems reward + model listed in their UGF inventory.

[LINUX] Install gltf-transform CLI on Oracle VM:

```
npm install -g @gltf-transform/cli
```

Prisma Schema

```
// Add to apps/server/prisma/schema.prisma
model UGFModel {
  id          String @id @default(cuid())
  userId      String
  name        String
  description String?
  r2Key       String // e.g.
  "ugf/clxyz123/model_abc.glb"
  status      String @default("PENDING") //
  PENDING | APPROVED | REJECTED
  triangles   Int
  fileSizeKb  Int
  submittedAt DateTime @default(now())
  reviewedAt  DateTime?
  reviewedBy  String? // modId who reviewed
  reviewNote  String? // rejection reason
  if REJECTED user User @relation(fields: [userId], references: [id],
  onDelete: Cascade) @@index([userId]) @@index([status]) }
```

Section 4 — Ethical

Monetization: HavenGems

HavenWorld's monetization model is designed around a single principle: **money should buy joy, not advantage**. Every design decision in this section is guided by that principle — and by the explicit rejection of the dark patterns that have eroded trust in the gaming industry.

4.1 Core Philosophy

Commitment	Details
100% free to play	Every game feature is accessible to free players. HavenGems buy cosmetics only.
No pay-to-win — ever	Gems never buy stats, speed boosts, fishing advantage, crafting shortcuts, or competitive edge.
No loot boxes	Direct purchase only. Players see exactly what they're buying before they pay.
No energy systems or timers	Play as long as you want. No artificial throttling to push gem purchases.
No FOMO pressure	Seasonal passes never expire. No countdown timers on cosmetics.
No ads — ever	No advertising of any kind, in any part of HavenWorld.
No subscriptions	Every purchase is a one-time, permanent unlock.
No data sales	Player data is never sold, licensed, or shared with advertisers.

4.2 Gem Bundle Pricing

Bundle Name	Gems	Price (USD)	Value	Best For
Starter Pack	100 gems	\$0.99	Baseline	Trying the system for the first time
Regular Pack	550 gems	\$4.99	+10% bonus	Casual supporter — one or two cosmetics

Bundle Name	Gems	Price (USD)	Value	Best For
Plus Pack	1,200 gems	\$9.99	+20% bonus	Most popular — full outfit + furniture set
Mega Pack	2,500 gems	\$19.99	+25% bonus	Dedicated supporter — seasonal pass + extras

Payment Processing — First Real Cost:

Stripe is required for gem purchases. Stripe charges 2.9% + \$0.30 per transaction (no monthly fee, no setup fee). A Stripe account requires a legal business entity or personal tax ID (SSN in the US). This is the first time HavenWorld touches real money — consult a tax professional before launch if you are in the US.

4.3 Gifting System

Players can gift gem bundles to friends directly. This is a powerful organic growth mechanic — gift recipients discover the gem system naturally.

Gift API Endpoint

```
// POST /api/gems/gift // Body: { toUserId: string, bundleId: string,
message?: string } router.post('/gems/gift', requireAuth, async (req, res)
=> { const { toUserId, bundleId, message } = req.body; const fromUser =
req.user; // Verify sender has enough gems const bundle =
GEM_BUNDLES[bundleId]; if (!bundle) return res.status(400).json({ error:
'Invalid bundle' }); // Deduct from sender (or use Stripe charge for
direct purchase gift) // Credit to recipient await prisma.$transaction([
prisma.user.update({ where: { id: toUserId }, data: { gems:
{ increment: bundle.gems } } }), prisma.gemTransaction.create({
data: { fromUserId: fromUser.id, toUserId, amount:
bundle.gems, type: 'GIFT', bundleId, message: message
?? null } }) ]); // Real-time notification via Socket.io
io.to(toUserId).emit('GEMS_GIFTED', { from: fromUser.username,
amount: bundle.gems, message: message ?? 'Enjoy! 🎁' });
res.json({ success: true }); });
```

4.4 Seasonal Cosmetic Passes (No Expiry Pressure)

Seasonal passes are themed cosmetic collections sold as a flat-price bundle. Unlike battle passes in other games, HavenWorld's passes **never expire** and are available for the full 90-day season with no countdown pressure.

Pass Design Principles

- **No time gates:** All cosmetics in a pass unlock immediately on purchase. No "unlock day by day."
- **No FOMO timer:** The pass is listed as "available through [date]" — not displayed with a countdown clock.
- **After-season availability:** Passes from previous seasons return to the shop once per year in a "Classics Sale" at full price — no artificial scarcity.
- **Flat price:** \$4.99 per pass. Players always know the cost before buying.

Prisma Schema

```
// apps/server/prisma/schema.prisma
model SeasonalPass {
  id          String @id @default(cuid())
  name        String // "Autumn Harvest Pass 2026"
  season      String // "autumn_2026"
  price       Int     // in cents: 499
  availableFrom DateTime
  availableTo  DateTime
  items       Json     // array of cosmetic item IDs included
  createdAt   DateTime @default(now())
  userPasses  UserPass[]
}
model UserPass {
  id          String @id @default(cuid())
  userId      String
  passId      String
  purchasedAt DateTime @default(now())
  user        User    @relation(fields: [userId], references: [id], onDelete: Cascade)
  pass        SeasonalPass @relation(fields: [passId], references: [id])
  @@unique([userId, passId]) // prevent duplicate purchases
}
```

4.5 HavenWorld Player Promise



The HavenWorld Player Promise

This is our binding commitment to every player. These are not aspirational goals — they are non-negotiable constraints on how HavenWorld operates. If we ever break any of these promises, players may request a full refund of unused HavenGems.

- We will **never** sell, license, or share your personal data with advertisers or third parties.
- We will **never** add pay-to-win mechanics — no gem purchase will ever give a competitive advantage.
- We will **never** add loot boxes or random-chance purchases. You will always know what you're buying.
- We will **never** lock existing free features behind a paywall.
- We will **never** use countdown timers, FOMO mechanics, or manufactured scarcity.
- We will **always** display the exact contents and price of every purchasable item.
- If HavenWorld ever permanently shuts down, we will provide a minimum 30-day notice and a full refund policy for all unused HavenGems at the time of announcement.

— *Daniel, HavenWorld Creator*

Section 5 — GDPR & Privacy

Compliance

GDPR applies to any service processing data of EU residents — regardless of where the server is located. Even as an indie game, HavenWorld must implement data subject rights. These endpoints are not optional. The good news: your existing Prisma cascade delete configuration already does most of the heavy lifting.

| 5.1 Data Deletion Request Flow

Players must be able to permanently delete their accounts and all associated data. This endpoint is the implementation of GDPR Article 17 (Right to Erasure).

Deletion Endpoint

[LINUX] Create route: DELETE /api/account/delete

```
// apps/server/src/routes/account.routes.ts router.delete('/account/delete',
requireAuth, async (req, res) => {  const { password } = req.body;  const
userId = req.user.id;    // Step 1: Confirm password before irreversible
action  const user = await prisma.user.findUnique({ where: { id:
userId } });  const valid = await bcrypt.compare(password,
user!.passwordHash);  if (!valid) return res.status(401).json({ error:
'Password incorrect' });  // Step 2: Anonymize PII immediately (keeps row
for fraud audit trail)  await prisma.user.update({    where: { id:
userId },    data: {      username: `deleted_user_${userId}`,      email:
`deleted_${userId}@havenworld.invalid`,      passwordHash: 'DELETED',
deletedAt: new Date()    }  });  // Step 3: Cascade delete via Prisma
relations (onDelete: Cascade handles most)  // Prisma will cascade: Avatar,
Inventory, InventoryItem, LoftItems,  // GuestbookEntry, FishCatch,
CraftingQueue, Pet, ModLog (as target), etc.  // All relations with
onDelete: Cascade fire automatically.  // Step 4: Clear Redis player state
const redisKeys = await redis.keys(`player:${userId}:*`);  if
(redisKeys.length) await redis.del(redisKeys);  await redis.del(`muted:${
userId}`);  // Step 5: Delete R2 avatar assets  const r2Objects = await
r2Client.send(new ListObjectsV2Command({    Bucket: process.env.R2_BUCKET,
Prefix: `avatars/${userId}/`  }));  if (r2Objects.Contents?.length)
{    await r2Client.send(new DeleteObjectsCommand({      Bucket:
process.env.R2_BUCKET,      Delete: {        Objects:
r2Objects.Contents.map(o => ({ Key: o.Key! }))      }    }  ));  }  //
Step 6: Send confirmation email via Resend  await
```

```
resend.emails.send({    from: 'noreply@havenworld.pages.dev',    to:
user!.email, // use pre-anonymization email    subject: 'Your HavenWorld
account has been permanently deleted',    html: '<p>Your account and all
associated data have been permanently deleted. Thank you for playing
HavenWorld.</p>'    });    // Disconnect active socket if present
io.to(userId).emit('ACCOUNT_DELETED');
io.sockets.sockets.get(userId)?.disconnect(true);    res.json({ success:
true, message: 'Account permanently deleted.' }); });
```

Soft-delete retention:

The anonymized row (

deleted_user_{id}

) is retained for 30 days for fraud prevention audit purposes, then hard-deleted by a scheduled cron job. This is compliant with GDPR recital 65 — fraud prevention is a legitimate interest.

5.2 Data Export Endpoint (Right to Access)

GDPR Article 15 grants users the right to receive a copy of all data held about them. This endpoint generates a downloadable JSON zip.

[MAC] Install archiver package:

```
pnpm add archiver pnpm add -D @types/archiver
```

Export Endpoint

```
// GET /api/account/export router.get('/account/export', requireAuth,
rateLimit({ windowMs: 86400000, max: 1 })), async (req, res) => {    const
userId = req.user.id;    // Gather all user data in parallel    const
[profile, inventory, transactions, guestbook, chatLogs,
achievements, fishCatches] = await Promise.all([    prisma.user.findUnique({
where: { id: userId }, select: {    username: true, email: true,
createdAt: true, role: true    })),
prisma.inventoryItem.findMany({ where: { inventory: { userId } } })),
prisma.gemTransaction.findMany({ where: { OR: [{ fromUserId: userId,
{ toUserId: userId } ] } })),    prisma.guestbookEntry.findMany({ where: { OR:
[{ authorId: userId }, { guestbook: { userId } } ] } })),
prisma.chatLog.findMany({ where: { userId, createdAt: { gte: new
Date(Date.now() - 90 * 86400000) } } })),
prisma.userAchievement.findMany({ where: { userId }, include: { achievement:
```

```

true } })),      prisma.fishCatch.findMany({ where: { session: { userId } } })
]);    // Build zip archive  const archive = archiver('zip', { zlib:
{ level: 9 } });    res.attachment(`havenworld-data-export-${userId}.zip`);
archive.pipe(res);    archive.append(JSON.stringify({ profile, inventory,
transactions,      guestbook, chatLogs, achievements, fishCatches }, null, 2),
{ name: 'havenworld-data.json' }  );    archive.append(      'This file
contains all data HavenWorld holds about your account.\n' +      'To request
deletion, visit Settings > Delete Account.\n' +      'For questions:
privacy@havenworld.pages.dev',      { name: 'README.txt' }  );    await
archive.finalize(); });

```

5.3 Cookie Consent Banner

HavenWorld uses only essential cookies (JWT session token, room state). No analytics or advertising cookies. The consent banner is required by EU law regardless of cookie type.

```

// apps/client/src/ui/CookieConsent.ts  export function initCookieConsent():
void {    // Don't show again if already accepted    if
(localStorage.getItem('cookie_consent')) return;    const wrapper =
document.createElement('div');    wrapper.style.cssText =
[      'position:fixed', 'bottom:0', 'left:0', 'right:0',
'background:#0b2545', 'color:#d0eaeb',      'padding:16px 24px', 'font-
family:Calibri,sans-serif',      'font-size:13px', 'z-index:9999',
'display:flex', 'align-items:center', 'gap:16px'    ].join(';');
wrapper.innerHTML = `      <p style="margin:0;flex:1">🍪 HavenWorld
uses <strong>essential cookies only</strong> for authentication      and
gameplay session management. No tracking or advertising cookies are used.
</p>      <button id="cookie-accept" style="
background:#0e7c7b;color:#fff;border:none;      padding:8px
20px;cursor:pointer;font-size:13px;font-weight:bold">      Accept & Continue
</button>      <a href="/privacy.html" style="color:#a8dadac;white-
space:nowrap">      Privacy Policy      </a>    `;
document.body.appendChild(wrapper);    document.getElementById('cookie-
accept')!.addEventListener('click', () =>
{      localStorage.setItem('cookie_consent', 'accepted');
wrapper.remove();    }); }

```

5.4 Privacy Policy Outline

Host a complete Privacy Policy at `apps/client/public/privacy.html`. Below is the required section outline. Consult a privacy attorney for your jurisdiction before publishing.

#	Section	Required Content
1	What We Collect	Username, email address, gameplay events (room visits, item purchases, fishing sessions), IP address (for fraud prevention), device/browser type.
2	Why We Collect It	Authentication and account management; gameplay functionality; abuse and fraud prevention; aggregate analytics (no individual tracking).
3	How Long We Keep It	Account data: retained until deletion request. Chat logs: 90 days. Server access logs: 30 days. Anonymized aggregate analytics: indefinitely.
4	Who We Share It With	No third-party data sales or licenses. Infrastructure providers: Cloudflare (CDN, edge), Oracle Cloud (hosting), Resend (transactional email). Each provider is bound by a DPA.
5	Your Rights (GDPR)	Right to access (export endpoint), right to correction (Settings page), right to erasure (delete endpoint), right to portability (JSON export). Response within 30 days.
6	Children's Privacy	HavenWorld is not directed at children under 13 (or 16 in EU). No knowing collection of data from minors.
7	Contact	privacy@havenworld.pages.dev — or the GitHub Security Advisory for sensitive disclosures.

#	Section	Required Content
8	Last Updated	Include date. Update this date whenever policy changes. Notify active users of material changes via email.

Section 6 — Series Wrap-Up: Complete Project Reference

This section is the definitive reference for the completed HavenWorld project. Bookmark it, print it, pin it to your wall. Every major artifact, every secret, every database table, and every Socket.io event in one place.

6.1 Master File Tree (Complete Project)

```

havenworld/ | — .github/ | | — workflows/ | | | — client.yml #
Cloudflare Pages deploy (on push to main) | | | — server.yml #
SSH deploy to Oracle VM (on push to main) | | | — ISSUE_TEMPLATE/ | | | —
bug_report.yml # Bug report form (Section 1.3) | | | —
feature_request.yml # Feature request form (Section 1.3) | | | —
dependabot.yml # Automated dependency updates | | | —
pull_request_template.md # Auto-populates PR descriptions (Section 1.4) | | | —
apps/ | | — client/ | | | — public/ | | | — manifest.json #
PWA manifest (Part 9A) | | | — sw.js # Service worker for
offline caching | | | — privacy.html # GDPR privacy policy
(Section 5.4) | | | — src/ | | | — engine/ | | | —
BabylonEngine.ts # Engine init, canvas setup | | | —
SceneManager.ts # Scene lifecycle, room transitions | | | —
AvatarController.ts # WASD movement, animation state machine | | | —
| — RoomRenderer.ts # Room mesh loading, lighting, shadows | | | —
| | — FurnitureSystem.ts # Furniture placement, pickup, R2 model load | | | —
| | | — XRManager.ts # WebXR / VR mode (Section 3.2) | | | —
| | | — ui/ | | | — ChatUI.ts # Chat panel, emoji,
moderation commands | | | — InventoryUI.ts # Inventory
grid, drag-to-equip | | | — ShopUI.ts # HavenGems
shop, seasonal passes | | | — PassportUI.ts # Player

```

```

passport, achievement stamps | | | | | CookieConsent.ts # GDPR
cookie banner (Section 5.3) | | | | | MiniMap.ts # Room
minimap overlay | | | | | audio/ | | | AudioEngine.ts
# Web Audio API synthesizer (Part 9B) | | | | | socket/ | | | |
| SocketClient.ts # Socket.io client singleton | | | | |
EventHandlers.ts # All client-side event handlers | | | | |
main.ts # App entry point | | | tests/ | | | |
| unit/ # Vitest unit tests (Part 7B) | | | | |
e2e/ # Playwright end-to-end tests (Part 7B) | | | | |
| perf/ # Babylon.js NullEngine benchmarks (Part 7B) | | | | | | | | | |
| | index.html | | | vite.config.ts | | package.json | |
| | server/ | | | src/ | | handlers/ | | | |
| auth.ts # Login, register, JWT refresh | | | | |
room.ts # Room join/leave, player list | | | | |
trade.ts # P2P item trading system | | | | |
fishing.ts # Fishing minigame logic | | | | |
crafting.ts # Crafting queue, recipe validation | | | | |
pets.ts # Pet state, feeding, tricks | | | | |
moderation.ts # MOD_ACTION handler (Section 2.1) | | | | |
ugf.ts # User-generated furniture upload | | | | |
middleware/ | | | | | auth.middleware.ts # JWT verification, ban
check | | | | | rateLimiter.ts # Redis-backed rate limiting
(Part 8) | | | | | sanitize.ts # Input sanitization, XSS
prevention | | | routes/ | | | auth.routes.ts
# /api/auth/* endpoints | | | account.routes.ts #
/api/account/export, /api/account/delete | | | shop.routes.ts
# /api/shop/*, /api/gems/* | | | admin.routes.ts #
/api/admin/* (invite, dashboard) | | | services/ | | | |
| RoomManager.ts # Room state, player presence | | | | |
EconomyService.ts # HavenGems, transactions, gifting | | | | |
NotificationService.ts # In-game + email notifications | | | | |
AnalyticsService.ts # DAU, event tracking (Part 9B) | | | | |
index.ts # Server entry, Express + Socket.io init | | | | |
| prisma/ | | | schema.prisma # Complete DB schema
(~34 models) | | | migrations/ # All historical
migrations | | | tests/ | | | unit/ #
Jest unit tests (Part 7A) | | | integration/ #
Supertest API tests (Part 7A) | | | load/ # k6
load test scripts (Part 7A) | | ecosystem.config.cjs # PM2
config (cluster mode) | | package.json | packages/ | |
shared/ | | src/ | | types/ | | | socket-
events.ts # Full Socket.io event type registry | | | | |
user.ts # User, Role, BanRecord types | | | | |
room.ts # Room, RoomItem, Furniture types | | | | |
fishing.ts # Fish catalog, rod types, rarity weights | | | | |
| crafting.ts # Recipes, material costs, output tables | | | | | |
| | pets.ts # Pet species, state machine, mood system | | | | |
| | pizza.ts # Pizza ingredient combos, scoring | | | | |
| | catalog.ts # Shop catalog, gem prices, seasonal flags | | | | |
| | package.json | scripts/ | create_partition.sh
# Quarterly PostgreSQL partition creator | db_backup.sh
# Automated pg_dump to R2 | health_check.sh # Full
stack health check (Part 10A) | CONTRIBUTING.md #
Contributor guide (Section 1.1) | CODE_OF_CONDUCT.md #
Community standards (Section 1.5) | SECURITY.md #
Vulnerability disclosure (Section 1.6) | LICENSE
# GPL-3.0 | README.md # Project overview, quick
start, badges | pnpm-workspace.yaml # Monorepo workspace
config | package.json # Root scripts: dev, build,
test, lint

```

6.2 Complete GitHub Secrets Checklist

Secret Name	Value Description	Used In Workflow
CLOUDFLARE_API_TOKEN	Cloudflare API token with Pages deploy permissions	client.yml
CLOUDFLARE_ACCOUNT_ID	Cloudflare account ID (found in dashboard sidebar)	client.yml
SSH_HOST	Oracle VM public IP address	server.yml
SSH_USERNAME	Oracle VM SSH user (typically ubuntu)	server.yml
SSH_PRIVATE_KEY	Private key matching Oracle VM's authorized_keys	server.yml
SSH_PORT	SSH port number (default: 22)	server.yml
DATABASE_URL	Full PostgreSQL connection string with credentials	server.yml
REDIS_URL	Redis connection string (localhost or socket path)	server.yml
JWT_SECRET	256-bit random secret for JWT signing — generate with <code>openssl rand -base64 32</code>	server.yml
RESEND_API_KEY	Resend transactional email API key	server.yml
R2_ACCOUNT_ID	Cloudflare R2 account ID	server.yml
R2_ACCESS_KEY_ID	R2 S3-compatible access key ID	server.yml
R2_SECRET_ACCESS_KEY	R2 S3-compatible secret access key	server.yml
VITE_API_URL	Public API base URL (e.g. <code>https://129-146-88-42.nip.io/api</code>)	client.yml

Secret Name	Value Description	Used In Workflow
VITE_SOCKET_URL	Public Socket.io URL (e.g. <code>https://129-146-88-42.nip.io</code>)	client.yml
VITE_R2_URL	Public R2 CDN base URL (e.g. <code>https://assets.havenworld.pages.dev</code>)	client.yml
ADMIN_INVITE_CODE	Secret code for first admin account registration (Part 4C)	server.yml
STRIPE_SECRET_KEY (Phase 4)	Stripe secret key for payment processing — add when HavenGems go live	server.yml

6.3 Master Socket.io Event Registry

Event Name	Direction	Payload Summary	Handler File
— AUTH EVENTS —			
AUTH_HANDSHAKE	Client → Server	{ token: string }	auth.ts
AUTH_SUCCESS	Server → Client	{ userId, username, role, gems }	auth.ts
AUTH_FAILED	Server → Client	{ reason: string }	auth.ts
BANNED	Server → Client	{ reason: string, banDate: string }	auth.middleware.ts
— ROOM EVENTS —			
ROOM_JOIN	Client → Server	{ roomId: string }	room.ts
ROOM_JOINED	Server → Client	{ roomId, players: PlayerData[], items: RoomItem[] }	room.ts
ROOM_PLAYER_JOIN	Server → Room	{ player: }	room.ts

Event Name	Direction	Payload Summary	Handler File
D		PlayerData }	
ROOM_PLAYER_LEFT	Server → Room	{ userId: string }	room.ts
ROOM_LEAVE	Client → Server	{ } (empty)	room.ts
— AVATAR EVENTS —			
AVATAR_MOVE	Client → Server	{ x, y, z, rotation, animation }	room.ts
AVATAR_MOVED	Server → Room	{ userId, x, y, z, rotation, animation }	room.ts
AVATAR_UPDATE	Client → Server	{ outfit, hairColor, skinTone, accessories }	room.ts
AVATAR_UPDATED	Server → Room	{ userId, avatarData }	room.ts
— CHAT EVENTS —			
CHAT_MESSAGE	Client → Server	{ message: string }	room.ts
CHAT_BROADCAST	Server → Room	{ userId, username, message, timestamp, role }	room.ts
CHAT_MUTED	Server → Client	{ reason: string }	moderation.ts
CHAT_EMOJI	Client → Server	{ emoji: string }	room.ts
— TRADE EVENTS —			
TRADE_REQUEST	Client → Server	{ targetUserId: string }	trade.ts
TRADE_INVITE	Server → Client	{ fromUserId, fromUsername }	trade.ts
TRADE_ACCEPT	Client → Server	{ tradeId: string }	trade.ts
TRADE_OFFER	Client → Server	{ tradeId, itemIds: string[], gems: number }	trade.ts
TRADE_CONFIRMED	Server → Both	{ tradeId, result: 'SUCCESS' }	trade.ts

Event Name	Direction	Payload Summary	Handler File
		'CANCELLED' }	
— FISHING EVENTS —			
FISHING_CAST	Client → Server	{ rodId: string, baitId?: string }	fishing.ts
FISHING_BITE	Server → Client	{ sessionId, difficulty: number }	fishing.ts
FISHING_REEL	Client → Server	{ sessionId, timing: number }	fishing.ts
FISHING_CATCH	Server → Client	{ fish: FishSpecies, weight, rarity, xpGained }	fishing.ts
FISHING_MISS	Server → Client	{ sessionId }	fishing.ts
— CRAFTING EVENTS —			
CRAFT_START	Client → Server	{ recipeId: string }	crafting.ts
CRAFT_QUEUED	Server → Client	{ queueId, completeAt: string }	crafting.ts
CRAFT_COMPLETE	Server → Client	{ queueId, itemId, itemName }	crafting.ts
— SHOP & GEMS EVENTS —			
SHOP_BUY	Client → Server	{ itemId: string, currency: 'GEMS' 'COINS' }	shop routes
SHOP_BUY_SUCCESS	Server → Client	{ itemId, newBalance, inventoryUpdate }	shop routes
GEMS_GIFTED	Server → Client	{ from: string, amount: number, message?: string }	EconomyService
GEMS_BALANCE_UPDATE	Server → Client	{ newBalance: number }	EconomyService
— PET EVENTS —			
PET_FEED	Client → Server	{ petId: string, foodItemId: string }	pets.ts

Event Name	Direction	Payload Summary	Handler File
PET_STATE_UPDATE	Server → Client	{ petId, hunger, happiness, energy, mood }	pets.ts
PET_TRICK	Client → Server	{ petId: string, trick: string }	pets.ts
PET_TRICK_RESULT	Server → Room	{ petId, trick, success: boolean }	pets.ts
— MODERATION EVENTS —			
MOD_ACTION	Client → Server	{ action, targetId, reason?, duration? }	moderation.ts
MOD_ACTION_CONFIRMED	Server → Mod	{ action, targetId }	moderation.ts
MOD_WARNING	Server → Client	{ message: string }	moderation.ts
MUTED	Server → Client	{ duration: number, reason: string }	moderation.ts
— LOFT EVENTS —			
LOFT_FURNITURE_PLACEMENT	Client → Server	{ itemId, x, y, z, rotation }	room.ts
LOFT_FURNITURE_PLACED	Server → Room	{ roomId, itemId, x, y, z, rotation, placedBy }	room.ts
LOFT_FURNITURE_REMOVE	Client → Server	{ roomId: string }	room.ts
LOFT_THEME_CHANGE	Client → Server	{ themeId: string }	room.ts
— PASSPORT & ACHIEVEMENT EVENTS —			
PASSPORT_VIEW	Client → Server	{ targetUserId: string }	room.ts
PASSPORT_DATA	Server → Client	{ user, achievements, stamps, fishCaught, joinedAt }	room.ts
ACHIEVEMENT_UNLOCKED	Server → Client	{ achievement: AchievementData }	NotificationService

Event Name	Direction	Payload Summary	Handler File
ACCOUNT_DELETED	Server → Client	{ } — triggers forced logout	account.routes.ts

6.4 Complete Prisma Schema Summary

Table	Key Fields	Purpose
User	id, username, email, passwordHash, role, gems, coins, bannedAt, deletedAt	Core player accounts and auth
Session	id, userId, token, expiresAt, ipAddress	JWT refresh token tracking
Room	id, name, type, ownerId, capacity, theme, isPublic	All room definitions (public + lofts)
RoomItem	id, roomId, itemId, placedBy, x, y, z, rotation	Furniture placed in rooms
Avatar	id, userId, skinTone, hairColor, hairStyle, r2TextureKey	Avatar visual configuration
AvatarCustomization	id, avatarId, slotType, itemId	Currently equipped outfit slots
Inventory	id, userId	Container for player items (1:1 with User)
InventoryItem	id, inventoryId, itemId, quantity, acquiredAt, source	Items owned by player
ShopItem	id, name, category, gemCost, coinCost, r2ModelKey, isLimited, available	Purchasable item catalog
ShopEpoch	id, name, startDate, endDate, featuredItemIds	Seasonal shop rotations
Transaction	id, userId, type, itemId, coinAmount, createdAt	Coin economy ledger

Table	Key Fields	Purpose
GemTransaction	id, fromUserId, toUserId, amount, type, bundleId, stripePaymentId	HavenGems purchase and gift ledger
FishingSession	id, userId, roomId, rodId, startedAt, endedAt	Fishing activity sessions
FishCatch	id, sessionId, speciesId, weight, rarity, caughtAt	Individual fish catch records
TradeListing	id, sellerId, itemId, askingGems, askingCoins, status, createdAt	Player trade marketplace listings
TradeOffer	id, listingId, buyerId, offeredItemIds, offeredGems, status	Offers on trade listings
CraftingQueue	id, userId, recipeId, startedAt, completeAt, claimed	Active and completed craft jobs
CraftingRecipe	id, name, outputItemId, durationSeconds, ingredientJson	Crafting recipe definitions
Pet	id, userId, speciesId, name, adoptedAt, isActive	Player-owned pets
PetState	id, petId, hunger, happiness, energy, lastFed, lastPlayed	Real-time pet welfare state
PizzaSession	id, roomId, startedAt, endedAt, score	Pizza Planet minigame sessions
PizzaOrder	id, sessionId, recipe, completedAt, accuracy	Individual pizza orders in session
Guestbook	id, userId	Guestbook container (1:1 with User)
GuestbookEntry	id, guestbookId, authorId, message, createdAt	Messages left on player guestbooks
ModLog	id, modId, targetId, action, reason, duration, createdAt	Permanent moderation audit trail
BanRecord	id, userId, reason, bannedBy, bannedAt, expiresAt, appealed	Detailed ban history

Table	Key Fields	Purpose
Achievement	id, name, description, category, iconKey, xpReward, gemReward	Achievement definitions
UserAchievement	id, userId, achievementId, unlockedAt	Player achievement unlocks
Passport	id, userId, bio, motto, viewCount	Player profile card
AchievementStamp	id, passportId, achievementId, slot, displayedAt	Stamps shown on passport (curated by player)
Neighborhood	id, name, theme, createdAt	Linked loft neighborhoods (Phase 4)
LoftSlot	id, neighborhoodId, userId, slotNumber, displayName	Player's street position in neighborhood
UGFModel	id, userId, name, r2Key, status, triangles, reviewedBy	User-generated furniture uploads (Phase 4)
SeasonalPass	id, name, season, price, availableFrom, availableTo, items	Seasonal cosmetic pass definitions
UserPass	id, userId, passId, purchasedAt	Player-owned seasonal passes
GameEvent	id, name, type, startDate, endDate, rewardJson, active	Seasonal / holiday event definitions
RendererPerf	id, userId, fps, drawCalls, gpuMs, timestamp, roomId	Client performance telemetry (Part 10A)
AnalyticsEvent	id, userId, event, properties, sessionId, timestamp	Gameplay analytics (Part 9B)

6.5 The 10-Part Series Complete Overview

Part	Title	Key Topics Covered
Competitive Analysis	Market Research & Game Design Brief	Club Penguin / Habbo Hotel analysis, monetization model research, feature prioritization, target audience, free infrastructure strategy
Part 1	Phase 0 Mac Setup	Node.js, pnpm, VS Code, GitHub repository, Oracle Cloud account creation, SSH key setup, monorepo scaffold
Part 2	Phase 0 Linux — Oracle Cloud VM	VM provisioning, Ubuntu setup, Nginx reverse proxy, PostgreSQL install, Redis install, Let's Encrypt SSL, firewall rules, nip.io domain
Part 3	Phase 1 Mac — Client Foundation	Vite + Babylon.js + TypeScript setup, Cloudflare Pages deploy, GitHub Actions CI/CD for client, PWA manifest, first 3D scene
Part 4A	Phase 1 Linux — Server Entry Point	Express 5 setup, Prisma ORM, full database schema, PostgreSQL migrations, TypeScript strict config, shared package setup
Part 4B	Phase 1 Linux — Auth & Socket.io	JWT authentication system, bcrypt password hashing, Socket.io integration, RoomManager, player presence, auth middleware
Part 4C	Phase 1 Linux — Admin & Deploy	Admin invite code system, admin dashboard endpoints, SSH GitHub Actions deploy pipeline, PM2 cluster mode, zero-downtime restart
Part 5A	Phase 2 — 3D Engine	Babylon.js scene setup,

Part	Title	Key Topics Covered
	Core	room mesh loading, camera system, lighting and shadows, WebGL optimization, Babylon Inspector integration
Part 5B	Phase 2 — Avatars, Blender & R2	Full avatar system, Blender 3D pipeline for avatar meshes, furniture placement system, Cloudflare R2 asset storage, CDN setup
Part 6	Phase 3 — All Legacy Features	All 11 gameplay systems: fishing, P2P trading, crafting, pets, Pizza Planet, guestbook, passport, achievements, neighborhoods (lite), mini-map, audio synthesizer
Part 7A	Server-Side Testing	Jest unit tests, Supertest API integration tests, Socket.io testing with socket.io-client, Prisma mock setup, k6 load testing scripts
Part 7B	Client-Side Testing	Vitest unit tests, Playwright E2E tests, Babylon.js NullEngine for headless rendering tests, performance benchmark suite
Part 8	Security Audit	Auth hardening, SQL injection prevention, XSS protection, rate limiting with Redis, Nginx security headers, OWASP review, dependency audit
Part 9A	Launch Readiness	Alpha/Beta player gates, PWA offline mode, R2 CDN optimization, zero-downtime deployment, beta feedback collection, launch checklist
Part 9B	Launch Monitoring & Live	Custom analytics

Part	Title	Key Topics Covered
	Ops	dashboard, Sentry error tracking, player onboarding flow, seasonal event system, patch notes system, DAU/MAU tracking
Part 10A	Scaling Beyond Free Tier	Horizontal Socket.io scaling with Redis adapter, PostgreSQL table partitioning, WebGPU migration path, CDN edge caching, cost thresholds, performance profiling
Part 10B	Community & Series Finale	Open-source health (CONTRIBUTING, CoC, issue templates), volunteer moderator program, Phase 4+ roadmap (mobile, VR, neighborhoods, UGF), ethical monetization, GDPR, complete project reference

Series Closing —

Congratulations, Daniel

Series Complete — HavenWorld Build Guide, Parts 1–10

From a blank terminal in Anoka, MN to a fully deployed, tested, secured, monitored, and community-ready 3D social virtual world — built by one person, on 100% free infrastructure, without a single dollar of VC funding.

"HavenWorld is proof that indie game development doesn't require VC funding — it requires vision, consistency, and the right roadmap."

| What You've Built

Look at what exists now. A browser-based 3D social virtual world with multiplayer rooms, a complete avatar system with Blender-crafted meshes, 11 fully implemented gameplay features (fishing, crafting, pets, trading, pizza minigame, guestbook, passport, achievements, mini-map, neighborhoods, audio synthesizer), a HavenGems economy with ethical monetization design, a volunteer moderator program with in-game tooling, a complete CI/CD pipeline, a full test suite, a security-hardened API, GDPR-compliant privacy tools, a scalable architecture ready for horizontal growth, and now an open-source community foundation with every document a contributor needs to join you.

The infrastructure cost: **\$0/month**. The first dollar will be spent if you choose to publish on the iOS App Store (\$99/yr). Everything else — Oracle VM, PostgreSQL, Redis, Nginx, Cloudflare Pages, Cloudflare R2, Resend, Discord — free tier, indefinitely.

| Final Launch Checklist

Before you call HavenWorld publicly launched, verify every item below:

- All 17 GitHub Actions secrets are set and workflows run without errors

- SSL certificate (Let's Encrypt) is valid, HTTPS enforces everywhere, and auto-renewal cron is verified
- At least 10 beta players have created accounts and completed the full onboarding flow
- Socket.io event loop lag is below 50ms under simulated 50-player load (k6 test)
- Cloudflare R2 avatar and furniture assets load in under 500ms on a standard connection
- All 11 Phase 3 features are functional and manually tested end-to-end
- GDPR data deletion endpoint tested — account fully anonymized and R2 assets removed
- GDPR data export endpoint tested — JSON zip contains all expected user data
- At least 2 volunteer moderators have been recruited, onboarded, and have working mod commands
- CONTRIBUTING.md, CODE_OF_CONDUCT.md, and SECURITY.md are live in the public repository
- Analytics dashboard shows DAU tracking and at least one week of baseline data
- Sentry error monitoring is active and at least one test error has been captured and resolved
- Privacy Policy is live at `/privacy.html` and cookie consent banner is functional
- README.md includes screenshots, a feature list, and a "play now" link

When you're ready:

Post your launch on [r/indiegaming](#), [r/gamedev](#), and [Hacker News \(Show HN\)](#).

HavenWorld's "100% free infrastructure, solo developer" story is genuinely remarkable and will resonate with those communities. Your README is your pitch

deck — make it count.

Series Complete — HavenWorld Build Guide, Parts 1-10.

Go build something amazing. 

HavenWorld Build Guide — Part 10B of 10 (SERIES FINALE) | Author: Daniel |
Anoka, MN | September 18, 2026

This document is part of a 10-part indie game development blueprint series. All infrastructure referenced is free tier as of document date.